Exemplu - Filtru

```
chan  input (char),
      output (char [1:Maxl]);

process car_linii{
  char line[1:Maxl]; int i=1;
  while (true) {
    receive input (line[i]);
    while (line[i] <> CR and i < Maxl) {
      /* line[1:i] = ultimele i caractere introduse*/
      i = i + 1;
      receive input (line[i]);
    }
    send output (line);
    i = 1;
  }
}
```

Exemplu - Replicated Workers (2)

```
chan sac (real a, b, fa, fb, aria);
chan rezultat (real a, b, aria);

process Administrator{
  real xl, xr, yl, yr, a, b, aria, total: real;
  /* alte variabile marcare intervale terminate */

  yl = f(xl); yr = f(xr);
  aria = (yl + yr) * (xr - xl) / 2;
  send sac(xl, xr, yl, yr, aria);

  while (nu s-a calculat toata aria) {
    receive rezultat (a, b, aria);
    total = total + aria;
    marchează intervalul [a, b] terminat;
  }
}
```

Exemplu - Replicated Workers (3)

```
process Lucru[i=1 to n]{
  real a, b, m, ya, yb, ym;
  real ariaSt, ariaDr, ariaTot, dif;
  while (true) {
    receive sac (a, b, ya, yb, ariaTot);
    m = (a + b) / 2;
    ym = f(m);
    calculează ariaSt și ariaDr;
    dif = ariaTot - ariaSt - ariaDr;
    if (dif mic) send rezultat (a, b, ariaTot);
    else { send sac (a, m, ya, ym, ariaSt);
           send sac (m, b, ym, yb, ariaDr);
    }
  }
}
```

Sincrona

Exemplu – comunicare producator / consumator

```
chan prodcons (real);

process prod {
   real date[n];
   for [i = 1 to n] {
      calculeaza date[i];
      synch_send prodcons (date[i]);    /* intarzie transmitatorul */
   }                                    /* daca receptorul este mai lent */
}
process cons {
   real buf[n];
   for [i = 1 to n] {
      receive prodcons (buf[i]);
      prelucreaza datele primite;
   }
}
```